

Codigo de la Interfaz en Python:

```
1 import threading
2 from datetime import datetime
3 import mysql.connector
4 import serial
5 from tkinter import messagebox
6 import tkinter as tk
7 from tkinter import ttk
8 from ttkthemes import ThemedTk
9
10 PUERTO = "/dev/ttyUSB0"
11 BAUDIOS = 9600
12 DB_HOST = "localhost"
13 DB_USER = "root"
14 DB_PASSWORD = "root"
15 DB_NAME = "rfid_registro"
16 DB_PORT = 3306
17
18 THEME = "equilux"
19 COLOR_ACCESO = "#9C0E6C"
20 COLOR_DENEGADO = "#541E75"
21 COLOR_CONSULTA = "#2930AC"
22 COLOR_SALIR = "#F10060"
23
24 TIEMPO_PUERTA = 5000
25
26 is_connected = False
27
28 arduino = serial.Serial()
29 arduino.port = PUERTO
30 arduino.baudrate = BAUDIOS
31 arduino.timeout = 0.5
32 arduino.write_timeout = 0.5
33
34 ventana_consulta = None
35 tree = None
36 lblCount = None
37 lblCargando = None
38 rango_var = None
39 entryBusqueda = None
40 timer_standby = None
41
42
43 def insertar_acceso(uuid, nombre, acceso_concedido):
44     try:
45         conexion = mysql.connector.connect(
46             host=DB_HOST,
47             user=DB_USER,
48             password=DB_PASSWORD,
49             database=DB_NAME,
50             port=DB_PORT,
51         )
52         cursor = conexion.cursor()
53         query = "INSERT INTO accesos (uuid, nombre, acceso_concedido, fecha)
54             VALUES (%s, %s, %s, %s)"
```

```

54     fecha = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
55     valores = (uuid, nombre, acceso_concedido, fecha)
56     cursor.execute(query, valores)
57     conexion.commit()
58     cursor.close()
59     conexion.close()
60     return True
61 except mysql.connector.Error as e:
62     print(f"Error MySQL: {e}")
63     return False
64
65
66 def ejecutar_query(rango, busqueda=""):
67     try:
68         conexion = mysql.connector.connect(
69             host=DB_HOST,
70             user=DB_USER,
71             password=DB_PASSWORD,
72             database=DB_NAME,
73             port=DB_PORT,
74         )
75         cursor = conexion.cursor()
76
77         if rango == "hora":
78             where = "WHERE fecha >= NOW() - INTERVAL 1 HOUR"
79         elif rango == "dia":
80             where = "WHERE DATE(fecha) = CURDATE()"
81         elif rango == "semana":
82             where = "WHERE YEARWEEK(fecha, 1) = YEARWEEK(CURDATE(), 1)"
83         elif rango == "mes":
84             where = "WHERE MONTH(fecha) = MONTH(CURDATE()) AND YEAR(fecha) = YEAR(CURDATE())"
85         else:
86             where = ""
87
88         query = f"SELECT uuid, nombre, acceso_concedido, fecha FROM accesos {where}"
89
90         if busqueda:
91             busqueda = busqueda.strip()
92             query += " AND (nombre LIKE %s OR uuid LIKE %s)"
93             query += " ORDER BY fecha DESC"
94             cursor.execute(query, (f"%{busqueda}%", f"%{busqueda}%"))
95         else:
96             query += " ORDER BY fecha DESC"
97             cursor.execute(query)
98
99         rows = cursor.fetchall()
100        cursor.close()
101        conexion.close()
102        return rows
103    except mysql.connector.Error as e:
104        print(f"Error MySQL: {e}")
105        return []
106
107

```

```

108 def poblar_treeview(rows):
109     global tree, lblCount, lblCargando
110     if tree is None:
111         return
112
113     for item in tree.get_children():
114         tree.delete(item)
115
116     for row in rows:
117         uuid, nombre, acceso, fecha = row
118         acceso_str = "ACCESO" if acceso else "DENEGADO"
119         nombre_str = nombre if nombre else "---"
120         fecha_str = (
121             fecha.strftime("%Y-%m-%d %H:%M:%S")
122             if isinstance(fecha, datetime)
123             else str(fecha)
124         )
125         tree.insert("", "end", values=(uuid, nombre_str, acceso_str, fecha_str))
126
127     lblCount.config(text=f"Total: {len(rows)} registros")
128     if lblCargando:
129         lblCargando.config(text="")
130
131
132 def cargar_registros_async(rango, busqueda=""):
133     global lblCargando
134     if lblCargando:
135         lblCargando.config(text="Cargando...")
136
137     def trabajo():
138         rows = ejecutar_query(rango, busqueda)
139         ventana_principal.after(0, poblar_treeview, rows)
140
141     threading.Thread(target=trabajo, daemon=True).start()
142
143
144 def on_rango_change():
145     global rango_var
146     rango = rango_var.get()
147     busqueda = ""
148     if entryBusqueda:
149         busqueda = entryBusqueda.get()
150     cargar_registros_async(rango, busqueda)
151
152
153 def on_buscar():
154     global rango_var
155     rango = rango_var.get()
156     busqueda = entryBusqueda.get() if entryBusqueda else ""
157     cargar_registros_async(rango, busqueda)
158
159
160 def abrir_ventana_consulta():
161     global ventana_consulta, tree, lblCount, lblCargando, rango_var,
    entryBusqueda

```

```

162
163     if ventana_consulta is not None and ventana_consulta.winfo_exists():
164         ventana_consulta.lift()
165         return
166
167     ventana_consulta = tk.Toplevel(ventana_principal)
168     ventana_consulta.title("Consultar Registros")
169     ventana_consulta.geometry("650x500")
170     ventana_consulta.configure(bg=bg_color)
171
172     rango_var = tk.StringVar(value="hora")
173
174     frame_rangos = ttk.Frame(ventana_consulta)
175     frame_rangos.pack(pady=10)
176
177     ttk.Radiobutton(
178         frame_rangos,
179         text="Ultima hora",
180         variable=rango_var,
181         value="hora",
182         command=on_rango_change,
183     ).pack(side=tk.LEFT, padx=10)
184     ttk.Radiobutton(
185         frame_rangos,
186         text="Hoy",
187         variable=rango_var,
188         value="dia",
189         command=on_rango_change,
190     ).pack(side=tk.LEFT, padx=10)
191     ttk.Radiobutton(
192         frame_rangos,
193         text="Semana",
194         variable=rango_var,
195         value="semana",
196         command=on_rango_change,
197     ).pack(side=tk.LEFT, padx=10)
198     ttk.Radiobutton(
199         frame_rangos,
200         text="Mes",
201         variable=rango_var,
202         value="mes",
203         command=on_rango_change,
204     ).pack(side=tk.LEFT, padx=10)
205
206     frame_busqueda = ttk.Frame(ventana_consulta)
207     frame_busqueda.pack(pady=5)
208
209     entryBusqueda = ttk.Entry(frame_busqueda, width=40)
210     entryBusqueda.pack(side=tk.LEFT, padx=5)
211     entryBusqueda.bind("<Return>", lambda e: on_buscar())
212
213     ttk.Button(
214         frame_busqueda, text="Buscar", command=on_buscar, style="Search.TButton
"
215     ).pack(side=tk.LEFT, padx=5)
216     ttk.Button(

```

```

217         frame_busqueda,
218         text="Limpiar",
219         command=lambda: [entryBusqueda.delete(0, tk.END), on_rango_change()],
220         style="Clear.TButton",
221     ).pack(side=tk.LEFT, padx=5)
222
223     lblCargando = ttk.Label(ventana_consulta, text="")
224     lblCargando.pack(pady=5)
225
226     columns = ("uuid", "nombre", "acceso", "fecha")
227     tree = ttk.Treeview(ventana_consulta, columns=columns, show="headings",
228 height=15)
229
230     tree.heading("uuid", text="UUID")
231     tree.heading("nombre", text="Nombre")
232     tree.heading("acceso", text="Acceso")
233     tree.heading("fecha", text="Fecha")
234
235     tree.column("uuid", width=120, anchor=tk.CENTER)
236     tree.column("nombre", width=100, anchor=tk.CENTER)
237     tree.column("acceso", width=80, anchor=tk.CENTER)
238     tree.column("fecha", width=150, anchor=tk.CENTER)
239
240     tree.pack(padx=10, pady=5, fill=tk.BOTH, expand=True)
241
242     lblCount = ttk.Label(ventana_consulta, text="Total: 0 registros")
243     lblCount.pack(pady=5)
244
245     frame_cerrar = ttk.Frame(ventana_consulta)
246     frame_cerrar.pack(pady=10)
247
248     ttk.Button(
249         frame_cerrar,
250         text="Cerrar",
251         command=ventana_consulta.destroy,
252         width=12,
253         style="Close.TButton",
254     ).pack()
255
256     ventana_consulta.protocol("WM_DELETE_WINDOW", lambda: [ventana_consulta.
257 destroy()])
258
259     cargar_registros_async("hora", "")
260
261 def procesar_linea(linea):
262     linea = linea.strip()
263     if linea == "READY":
264         ventana_principal.after(0, lambda: lbConexion.config(text="Conectado"))
265         return
266
267     if linea.startswith("ACCESO:"):
268         partes = linea.split(":")
269         if len(partes) == 3:
270             uuid = partes[1]
271             nombre = partes[2]

```

```

271         insertar_acceso(uuid, nombre, True)
272         ventana_principal.after(
273             0, lambda u=uuid, n=nombre: actualizar_gui(u, n, True)
274         )
275
276     elif linea.startswith("DENEGADO:"):
277         partes = linea.split(":")
278         if len(partes) >= 2:
279             uuid = partes[1]
280             insertar_acceso(uuid, "Desconocido", False)
281             ventana_principal.after(0, lambda u=uuid: actualizar_gui(u, "",
False))
282
283     elif linea == "CERRADO":
284         pass
285
286
287 def actualizar_gui(uuid, nombre, acceso):
288     global timer_standby
289
290     if timer_standby is not None:
291         ventana_principal.after_cancel(timer_standby)
292         timer_standby = None
293
294     if acceso:
295         lbEstado.config(text=f"ACCESO: {nombre}", style="Access.TLabel")
296         lbUUID.config(text=f"UUID: {uuid}")
297         timer_standby = ventana_principal.after(TIEMPO_PUERTA, volver_standby)
298     else:
299         lbEstado.config(text=f"DENEGADO", style="Denied.TLabel")
300         lbUUID.config(text=f"UUID: {uuid}")
301
302     hora = datetime.now().strftime("%H:%M:%S")
303     if acceso:
304         texto = f"[{hora}] ACCESO: {nombre} ({uuid})"
305     else:
306         texto = f"[{hora}] DENEGADO: {uuid}"
307     lbLog.config(text=texto)
308
309
310 def volver_standby():
311     global timer_standby
312     timer_standby = None
313     lbEstado.config(text="Esperando...", style="")
314     lbUUID.config(text="UUID: ---")
315
316
317 def DataReceivedHandler():
318     global is_connected
319     while is_connected:
320         try:
321             if arduino.is_open and arduino.in_waiting > 0:
322                 linea = arduino.readline().decode("utf-8", errors="ignore").
strip()
323
324                 if linea:
325                     procesar_linea(linea)

```

```

325         except serial.SerialException:
326             ventana_principal.after(
327                 0, lambda: messagebox.showerror("Error", "Conexion serial
perdida")
328             )
329             break
330         except Exception as e:
331             print(f"Error lectura: {e}")
332
333
334 def btnConectar_Click():
335     global is_connected
336     try:
337         if not arduino.is_open:
338             arduino.open()
339
340         btnConectar.config(state=tk.DISABLED)
341         btnDesconectar.config(state=tk.NORMAL)
342         lbConexion.config(text="Conectando...")
343
344         if not is_connected:
345             is_connected = True
346             hilo = threading.Thread(target=DataReceivedHandler, daemon=True)
347             hilo.start()
348
349         except serial.SerialException:
350             messagebox.showerror("Error", f"No se pudo abrir {PUERTO}")
351             lbConexion.config(text="Desconectado")
352
353
354 def btnDesconectar_Click():
355     global is_connected
356     is_connected = False
357
358     btnConectar.config(state=tk.NORMAL)
359     btnDesconectar.config(state=tk.DISABLED)
360
361     if arduino.is_open:
362         arduino.close()
363
364     lbConexion.config(text="Desconectado")
365     lbEstado.config(text="Esperando...")
366     lbUUID.config(text="UUID: ---")
367
368
369 def on_closing():
370     global is_connected
371     is_connected = False
372
373     if arduino.is_open:
374         arduino.close()
375
376     if ventana_consulta is not None and ventana_consulta.winfo_exists():
377         ventana_consulta.destroy()
378
379     ventana_principal.quit()

```

```

380     ventana_principal.destroy()
381
382
383 ventana_principal = ThemedTk(theme=THEME)
384 ventana_principal.title("Control de Acceso RFID")
385 ventana_principal.geometry("400x350")
386 bg_color = ttk.Style().lookup("TFrame", "background")
387 ventana_principal.configure(bg=bg_color)
388
389 style = ttk.Style()
390 style.configure("Connect.TButton", background=COLOR_ACCESO, foreground="white")
391 style.configure("Disconnect.TButton", background=COLOR_DENEGADO, foreground="
    white")
392 style.configure("Consult.TButton", background=COLOR_CONSULTA, foreground="white
    ")
393 style.configure("Exit.TButton", background=COLOR_SALIR, foreground="white")
394 style.configure("Search.TButton", background=COLOR_ACCESO, foreground="white")
395 style.configure("Clear.TButton", background=COLOR_DENEGADO, foreground="white")
396 style.configure("Close.TButton", background=COLOR_SALIR, foreground="white")
397 style.configure("Access.TLabel", foreground=COLOR_ACCESO)
398 style.configure("Denied.TLabel", foreground=COLOR_DENEGADO)
399
400 frame_header = ttk.Frame(ventana_principal)
401 frame_header.pack(pady=10)
402
403 lbConexion = ttk.Label(frame_header, text="Desconectado", font=("Arial", 12, "
    bold"))
404 lbConexion.pack()
405
406 lbEstado = ttk.Label(ventana_principal, text="Esperando...", font=("Arial", 16,
    "bold"))
407 lbEstado.pack(pady=10)
408
409 lbUUID = ttk.Label(ventana_principal, text="UUID: ---", font=("Arial", 11))
410 lbUUID.pack()
411 lbLog = ttk.Label(ventana_principal, text="", font=("Arial", 9))
412 lbLog.pack(pady=5)
413
414 frame_botones = ttk.Frame(ventana_principal)
415 frame_botones.pack(pady=15)
416
417 btnConectar = ttk.Button(
418     frame_botones,
419     text="Conectar",
420     command=btnConectar_Click,
421     width=12,
422     style="Connect.TButton",
423 )
424 btnConectar.grid(row=0, column=0, padx=5)
425
426 btnDesconectar = ttk.Button(
427     frame_botones,
428     text="Desconectar",
429     command=btnDesconectar_Click,
430     width=12,
431     state=tk.DISABLED,

```



```

432     style="Disconnect.TButton",
433 )
434 btnDesconectar.grid(row=0, column=1, padx=5)
435
436 btnConsultar = ttk.Button(
437     frame_botones,
438     text="Consultar",
439     command=abrir_ventana_consulta,
440     width=12,
441     style="Consult.TButton",
442 )
443 btnConsultar.grid(row=0, column=2, padx=5)
444
445 frame_salir = ttk.Frame(ventana_principal)
446 frame_salir.pack(pady=10)
447
448 btnSalir = ttk.Button(
449     frame_salir, text="Salir", command=on_closing, width=20, style="Exit.
450     TButton"
451 )
452 btnSalir.pack()
453
454 ventana_principal.protocol("WM_DELETE_WINDOW", on_closing)
455 ventana_principal.mainloop()
456 % --- FIN DEL CODIGO ---

```